Revised Project Strategy: Hyper-Local E-Commerce Platform

This document outlines the enhanced strategy for the localized e-commerce platform, focusing on robust, real-time stock management, refined customer discovery, and the definition of the third-party logistics (3PL) model.

1. Shopkeeper Stock Management (Real-Time Focus)

To ensure high data accuracy and prevent customer disappointment from ordering out-of-stock items, the platform will implement a dual-path approach to stock management.

A. Dedicated In-App Inventory Management System (Primary UI)

The platform will provide a user-friendly, centralized dashboard where shopkeepers can directly manage their inventory, which will be stored in the **Supabase** database.

Feature	Description	Technical Implementation
Simple Data Entry	Shopkeepers can quickly add, edit, or delete products and update the quantity of items in stock using a clean form interface (minimum of Name and Price required).	React forms linked to Spring Boot APIs for `POST/PUT/DELETE` operations on the Supabase inventory tables.
Real-Time Quantity Tracking	The quantity field will automatically decrement upon a customer placing an unconfirmed order and decrement permanently upon order finalization/payment.	Database triggers/functions in Supabase or immediate API calls from Spring Boot ensuring transactional integrity.
Bulk Upload/Update	Shopkeepers can download a standard Excel/CSV template and upload a file to perform bulk updates or initial stock loading. This is ideal for shops with large inventories.	Backend Spring Boot service processes the uploaded file, validates the data format (product ID, quantity, price), and updates the Supabase records in batches.
Low-Stock Alerts	Automated push notifications (via the app or SMS) to the shopkeeper when any product quantity drops below a customizable threshold (e.g., 5 units).	Supabase/Spring Boot cron job or real-time database listener checking inventory tables periodically against defined reorder points.

B. External Software Integration (Advanced Feature)

For shopkeepers who already use sophisticated Point of Sale (POS) or dedicated inventory software that exports to Excel/CSV, we will facilitate simplified data synchronization.

- **Excel/CSV Connector:** The shopkeeper will grant permission (or manually upload) a standardized file format. The system will process this file, map the shopkeeper's columns to the platform's required fields (Name, Price, Quantity), and update the live stock data.
- **Future API Integration (Phase 2):** While direct, real-time synchronization with disparate local POS systems is technically challenging, the architecture will be designed for future API expansion (Spring Boot) to allow direct data exchange with popular inventory management software (e.g., Zoho Inventory, QuickBooks) if the business scales.

2. Customer Experience and Shop Discovery

The customer-facing application will prioritize fast and relevant shop discovery based on the user's hyper-local context.

A. Advanced Search and Filtering

Feature	Description	UI/UX Component
Location-Based Filtering	Shops can be filtered instantly using a $\mathbf{0-20}\ \mathrm{km}$ radius, with granular sliders or preset buttons (e.g., **0-5 km, 5-10 km, 10-20 km**).	Interactive **Distance Slider** in the React application that updates the query parameters sent to the Spring Boot API (which then queries Supabase based on geo-location data).
Categorical Tags	Users can select multiple categories (Dairy, Grocery, Fashion, etc.) to refine results.	Filter panel using the vivid **Electric Blue** for category tags, allowing multi-select.
Specific Product Search	Users can search for a specific product name (e.g., "Amul Butter") and see results across all relevant shops within the operating radius, sorted by distance.	Main search bar utilizes Spring Boot's indexing/search functionality to perform fast lookups across all vendor inventory tables.
Shop Rating and Reviews	A simple $\mathbf{5-star}$ rating system and customer reviews will be displayed prominently to build trust and aid decision-making.	Star ratings displayed below the shop name and a dedicated reviews tab, influencing search ranking.
Featured Lists	Prominent sections on the homepage for curated lists.	**"Top Sellers (Local Bests),"** **"New Shops,"** and **"Exclusive Local Brands."**

B. Fulfillment Options

The three options for order receipt remain central to the platform's flexible model:

1. **Self-Collect:** Customer manually picks up the order at a scheduled time (saving their waiting time).
2. **Scheduled Pickup:** Customer sets a specific time slot during shop hours.
3. **Home Delivery (3PL):** Order is processed and fulfilled by a third-party logistics provider.

3. Technology Stack and Logistics Model

Technology Stack

- **Frontend: React + Vite** (UI/UX designed with Electric Blue and Warm Orange CTAs)
- **Backend: Java Spring Boot** (Robust, scalable API layer for business logic and data processing)
- **Database: Supabase** (PostgreSQL database with built-in real-time features for inventory and notifications)

Logistics Model: Third-Party Integration (3PL)

The platform will utilize a **Third-Party Logistics (3PL)** integration model for all home delivery services.

- **Integration Point:** The Spring Boot backend will serve as the integration layer, establishing APIs (likely using a REST or webhook model) to communicate with selected local 3PL providers.
- **Process Flow:**
  1. Customer finalizes an order and pays (Online or COD).
  2. Spring Boot sends an API request containing the shop location, customer delivery address, and package details to the 3PL partner.
  3. The 3PL system assigns a delivery agent.
  4. The platform receives real-time tracking links and status updates from the 3PL, which are relayed to the customer interface via the Supabase real-time subscription.

This approach allows the platform to focus on its core strength—connecting shops and buyers—while leveraging the specialized expertise and scale of logistics companies for delivery execution.